Pipeline di controllo qualità e outlier detection

1. Scopo del progetto

Questo progetto costruisce una pipeline riproducibile per controllare la qualità dei dati clinici e identificare osservazioni anomale nel dataset ospedaliero.

La pipeline non ha lo scopo di eliminare automaticamente i pazienti “strani”. Distingue tre situazioni diverse:

1. errori o valori chiaramente impossibili;
2. valori statisticamente estremi in una singola variabile;
3. pazienti rari rispetto alla combinazione di più variabili.

L'obiettivo principale è evitare che valori impossibili contaminino le feature aggregate del matricione. Se, per esempio, una frequenza cardiaca registrata come `978` entra nella media, nel massimo o nella deviazione standard di un paziente, le feature aggregate risultano falsate. Per questo la pulizia dei dati elementari deve avvenire prima delle aggregazioni.

2. Struttura dei file

```
progetto/
├── main.py
└── outlier_functions/
    ├── __init__.py
    ├── physiological_ranges.py
    └── outlier_pipeline.py
```

File	Funzione
`main.py`	Carica il matricione prodotto dal codice di Filippo e richiama Parte B e Parte C
`outlier_functions/physiological_ranges.py`	Contiene limiti fisiologici e configurazioni dei feature set
`outlier_functions/outlier_pipeline.py`	Contiene le funzioni operative delle Parti A, B e C
`outlier_functions/__init__.py`	Rende disponibili le funzioni come modulo Python

Il file `main.py` non contiene la logica interna degli algoritmi. Coordina soltanto le chiamate alle funzioni e il salvataggio dei risultati.

3. Idea generale della pipeline

Il flusso complessivo è:

```
Dati elementari di triage e sangue
        |
        v
Parte A: pulizia dei valori impossibili
        |
        v
Codice di Filippo: aggregazioni e feature engineering
        |
        v
Matricione finale
        |
        v
Parte B: outlier univariati con MAD
        |
        v
Parte C: outlier multivariati con DBSCAN
        |
        v
Matricione finale con flag e report
```

La Parte A viene richiamata dal codice di Filippo prima della costruzione del matricione. Il `main.py` richiama invece Parte B e Parte C dopo che il matricione è stato restituito.

4. Differenza tra le tre parti

Parte	Domanda	Modifica i valori?	Risultato
A	Il valore è chiaramente impossibile?	Sì, solo per gli impossibili: diventano NaN	Flag di qualità e report
B	Il valore è statisticamente estremo nella singola feature?	No	Flag univariata e robust z-score
C	La combinazione di feature è rara?	No	Cluster DBSCAN e flag di bassa densità

È fondamentale non confondere questi livelli. Un valore clinicamente grave può essere raro ma reale; per questo le Parti B e C non cancellano automaticamente i dati.

PARTE A — Pulizia pre-matricione

5. Obiettivo della Parte A

La Parte A identifica valori fuori dai limiti di plausibilità configurati. Un valore fuori da questi limiti viene considerato non affidabile per le aggregazioni e viene trasformato in `NaN`.

La Parte A non identifica la “normalità clinica”. Un paziente può avere una pressione molto alta o una frequenza cardiaca molto bassa e il dato può essere reale. Le soglie sono quindi ampie e

servono a separare gli errori evidenti dai casi clinici estremi.

Esempi

Valore	Azione	Motivo
`heartrate = 978`	Diventa `NaN`	Fuori dal limite di plausibilità
`o2sat = 103`	Diventa `NaN`	Una percentuale non può superare 100
`dbp = -57`	Diventa `NaN`	Pressione negativa non plausibile
`heartrate = 160`	Rimane nel dataset	Estremo ma possibile
`sbp = 220`	Rimane nel dataset	Grave ma possibile

6. Funzione principale della Parte A

```
clean_raw_measurements(
    df,
    physiological_ranges=None,
    id_column="hadm_id",
)
```

Parametri

- `df`: dataframe contenente le misure elementari;
- `physiological_ranges`: dizionario con `{nome_colonna: (minimo, massimo)}`; se non specificato usa i limiti di default;
- `id_column`: colonna identificativa, normalmente `hadm_id`.

Output

La funzione restituisce un oggetto `CleaningResult`:

```
result = clean_raw_measurements(triage_df)

cleaned_df = result.dataframe
cleaning_report = result.report
```

L'oggetto contiene:

- `result.dataframe`: dataframe pulito;
- `result.report`: report per ogni feature;
- `result.available_columns`: colonne configurate presenti nel dataframe;
- `result.missing_expected_columns`: colonne configurate ma assenti.

7. Logica interna della Parte A

`validate_input_dataframe()`

Controlla che:

- l'input sia un `pandas.DataFrame`;
- il dataframe non sia vuoto;
- la colonna identificativa, se presente, non sia completamente composta da `NaN`;
- le colonne configurate siano presenti o assenti.

Restituisce una copia del dataframe, così l'originale non viene modificato accidentalmente.

`clean_impossible_values()`

Per ogni colonna presente nella configurazione:

1. converte i valori in numerico con `pd.to_numeric(..., errors="coerce")`;
2. trasforma stringhe non interpretabili in `NaN`;
3. crea una maschera booleana per i valori sotto il limite minimo o sopra il limite massimo;
4. crea la flag `<feature>_invalid`;
5. sostituisce gli elementi invalidi con `NaN`;
6. registra conteggi e percentuali nel report.

Non vengono eliminate righe e non vengono sostituiti i valori con media o mediana.

`create_quality_flags()`

Somma le flag per-feature e crea:

```
n_invalid_values
has_invalid_value
```

`n_invalid_values` indica quanti valori impossibili erano presenti in una riga.
`has_invalid_value` vale 1 se almeno un valore era impossibile.

8. Limiti fisiologici

I limiti sono conservati nel file separato `physiological_ranges.py`. Questo permette di modificarli senza cambiare la logica della pipeline.

Triage

Feature	Unità	Minimo	Massimo
`temperature`	°F	80	115
`heartrate`	bpm	10	300
`resprate`	atti/min	4	80
`o2sat`	%	50	100
`sbp`	mmHg	40	300
`dbp`	mmHg	20	200
`pain`	scala	0	15

Blood test

Feature	Unità	Minimo	Massimo
`Hemoglobin`	g/dL	2	25
`White Blood Cells`	10^9/L	0.5	60
`Platelet Count`	10^9/L	5	1200
`C-Reactive Protein`	mg/L	0	600
`Creatinine`	mg/dL	0.1	20
`Sodium`	mEq/L	100	180
`Potassium`	mEq/L	1.5	9
`Glucose`	mg/dL	20	800

Questi valori devono essere verificati con le unità effettive dei file. Se Filippo usa nomi diversi o unità diverse, va modificato il file dei range prima di eseguire la Parte A.

9. Integrazione della Parte A nel codice di Filippo

Nel codice di Filippo, la funzione di costruzione del matricione deve richiamare la Parte A prima delle aggregazioni:

```
from outlier_functions.outlier_pipeline import clean_raw_measurements
from outlier_functions.physiological_ranges import (
    PHYSIOLOGICAL_RANGES_TRIAGE,
    PHYSIOLOGICAL_RANGES_BLOOD,
)


def crea_matricione():
    triage_df = pd.read_csv("Triage.csv")
    blood_df = pd.read_csv("BloodTest.csv")
```

```
    triage_clean = clean_raw_measurements(
        triage_df,
        physiological_ranges=PHYSIOLOGICAL_RANGES_TRIAGE,
    ).dataframe

    blood_clean = clean_raw_measurements(
        blood_df,
        physiological_ranges=PHYSIOLOGICAL_RANGES_BLOOD,
    ).dataframe

    # Il codice di aggregazione deve usare questi dataframe puliti.
    # matrix_df = aggrega(triage_clean, blood_clean)
    # return matrix_df
```

Il punto decisivo è che le medie, le deviazioni standard, i minimi e i massimi devono essere calcolati a partire da `triage_clean` e `blood_clean`, non dai dataframe originali.

PARTE B — Outlier univariati

10. Obiettivo della Parte B

La Parte B analizza una feature alla volta e cerca valori statisticamente molto lontani dalla distribuzione della feature stessa.

A differenza della Parte A, non modifica il valore. Un valore estremo può essere clinicamente reale; quindi viene aggiunta una flag, ma il dato originale resta disponibile.

11. Perché si usa il MAD

Il metodo usa:

```
mediana
MAD = Median Absolute Deviation
robust z-score
```

La formula del MAD è:

$$MAD = \text{mediana}(|x_i - \text{mediana}(x)|)$$

La formula dello robust z-score è:

$$z_i^{robusto} = \frac{0.6745(x_i - \text{mediana}(x))}{MAD}$$

Mediana e MAD sono meno influenzati dagli estremi rispetto a media e deviazione standard. Per questo sono adatti a dati clinici che possono avere distribuzioni asimmetriche.

12. Funzione principale della Parte B

```
apply_univariate_outlier_detection(
    df,
    exclude_columns=None,
    mad_threshold=3.5,
    min_observations=10,
)
```

Parametri

- `df`: matricione finale;
- `exclude_columns`: colonne da escludere, come identificativi, target, date numeriche e flag;
- `mad_threshold`: soglia dello robust z-score; default `3.5`;
- `min_observations`: minimo numero di valori non mancanti per analizzare una feature; default `10`.

Utilizzo

```
result_b = apply_univariate_outlier_detection(
    matrix_df,
    exclude_columns=["hadm_id", "ICD chapter"],
    mad_threshold=3.5,
    min_observations=10,
)

matrix_with_b = result_b.dataframe
univariate_report = result_b.report
```

13. Funzioni interne della Parte B

`compute_median()`

Calcola la mediana ignorando i `NaN`.

`compute_mad()`

Calcola la mediana delle deviazioni assolute dalla mediana.

`compute_robust_zscore()`

Trasforma ogni valore in uno score robusto. Se il MAD è zero, lo score non è calcolabile.

`detect_univariate_outliers()`

Esegue l'analisi su una lista di colonne esplicita. Per ogni feature aggiunge:

```
<feature>_univariate_outlier
<feature>_robust_zscore
```

Alla fine aggiunge:

```
n_univariate_outliers
has_univariate_outlier
```

14. Perché escludere alcune colonne

`hadm_id` è un identificativo numerico ma non è una misura clinica. Se venisse analizzato, il suo valore statistico non avrebbe significato.

Anche target, date convertite in numeri, codici e flag già costruite non devono entrare automaticamente nell'analisi. Le colonne da escludere vanno specificate nel `main.py`.

PARTE C — Outlier multivariati con DBSCAN

15. Obiettivo della Parte C

La Parte C considera più feature contemporaneamente. Un paziente può non essere estremo in una singola variabile, ma può risultare raro per la combinazione di tutte le variabili.

Esempio:

```
heartrate = 150 bpm
sbp = 90 mmHg
temperature = 103 °F
resprate = 35 atti/min
o2sat = 88%
```

Ogni valore può essere possibile individualmente, ma la combinazione può trovarsi in una regione poco densa della popolazione.

16. Perché DBSCAN

DBSCAN è un algoritmo basato sulla densità. Cerca regioni con molti punti vicini e identifica come `noise` i punti isolati.

Non richiede di fissare in anticipo il numero di cluster. Tuttavia, il risultato dipende dai parametri `eps`, `min_samples` e dalle feature scelte; per questo è un metodo esplorativo da calibrare sui dati reali.

Un punto `noise` non è automaticamente un errore clinico. È un caso raro secondo le variabili inserite nell'analisi.

17. Scaling con RobustScaler

Le feature hanno unità e scale differenti. La pressione è in mmHg, la saturazione è una percentuale e l'emoglobina è in g/dL.

DBSCAN usa distanze: senza scaling, le variabili numericamente più grandi dominerebbero la distanza. La pipeline applica `RobustScaler`, che usa mediana e intervallo interquartile.

Questo mantiene la comparabilità tra feature e limita l'influenza degli estremi.

18. Feature set DBSCAN

Il file `physiological_ranges.py` definisce tre configurazioni:

```
FEATURE_SETS = {
    "triage": [...],
    "triage_plus_blood": [...],
    "blood_only": [...],
}
```

`triage`

Usa:

```
temperature
heartrate
resprate
o2sat
sbp
dbp
```

È il set consigliato per iniziare perché è piccolo, interpretabile e temporalmente omogeneo.

`triage_plus_blood`

Aggiunge:

```
Hemoglobin
White Blood Cells
Platelet Count
C-Reactive Protein
```

Va usato solo se gli esami del sangue sono compatibili con il momento temporale dell'analisi e se il numero di righe complete rimane sufficiente.

`blood_only`

Usa solo le variabili ematiche configurate.

19. Funzione principale della Parte C

```
apply_dbscan_outlier_detection(
    df,
    feature_set="triage",
    eps=1.5,
    min_samples=10,
    max_missing_rate=0.30,
)
```

Parametri

`df`

Matricione finale. Può contenere anche le flag della Parte B, ma tali flag non vengono usate da DBSCAN se non sono incluse nel feature set.

`feature_set`

Può essere:

```
"triage"
"triage_plus_blood"
"blood_only"
```

`eps`

Raggio di vicinato dopo il `RobustScaler`.

- più basso: algoritmo più severo, più `noise`;
- più alto: algoritmo più permissivo, meno `noise`.

Il valore `1.5` è solo un punto di partenza.

`min_samples`

Numero minimo di punti vicini per definire una regione densa.

- valore più basso: cluster più facili da formare;
- valore più alto: metodo più conservativo.

Il valore `10` è iniziale e va calibrato.

`max_missing_rate`

Percentuale massima di missing consentita in una feature. `0.30` significa 30%.

Una feature oltre la soglia viene esclusa dal particolare esperimento DBSCAN. Non viene eliminata dal dataframe originale.

20. Gestione dei missing nella Parte C

DBSCAN non accetta `NaN`. Dopo la selezione delle feature, la pipeline usa per il clustering solo le righe complete rispetto alle feature selezionate.

Le righe non complete restano nel dataframe finale e ricevono:

```
dbscan_evaluable = 0
dbscan_cluster = NaN
dbscan_low_density = NaN
```

Per le righe valutabili:

```
dbscan_evaluable = 1
dbscan_cluster >= 0  -> appartiene a un cluster
dbscan_cluster = -1 -> noise
dbscan_low_density = 0 -> cluster
dbscan_low_density = 1 -> noise
```

21. Funzioni interne della Parte C

`select_dbscan_features()`

Seleziona le colonne richieste, controlla che esistano e rimuove quelle troppo incomplete. Restituisce anche le motivazioni delle esclusioni.

`prepare_dbscan_matrix()`

Converte le feature in numeriche, seleziona le righe complete e applica `RobustScaler`.

`detect_multivariate_outliers_dbscan()`

Esegue il modello DBSCAN, assegna le label e costruisce il report.

`apply_dbscan_outlier_detection()`

È il wrapper semplificato da utilizzare nel `main.py`.

MAIN E FLUSSO OPERATIVO

22. Ruolo del `main.py`

Il `main.py` deve rimanere semplice. Non deve contenere l'implementazione di MAD, DBSCAN o dei limiti fisiologici.

Il suo compito è:

1. importare la funzione di Filippo;
2. chiamare Filippo per ottenere il matricione;
3. richiamare Parte B;
4. richiamare Parte C;
5. salvare il risultato e i report.

23. Esempio di `main.py`

```
import pandas as pd

from pipeline_filippo import crea_matricione
from outlier_functions.outlier_pipeline import (
    apply_univariate_outlier_detection,
    apply_dbscan_outlier_detection,
)


MATRICIONE_OUTPUT = "matricione_con_outlier_flags.csv"
EXCLUDE_COLUMNS = ["hadm_id", "ICD chapter"]
DBSCAN_FEATURE_SET = "triage"
DBSCAN_EPS = 1.5
DBSCAN_MIN_SAMPLES = 10
DBSCAN_MAX_MISSING_RATE = 0.30


def main():
    # Filippo restituisce il matricione già costruito.
    # La sua funzione deve avere già richiamato Parte A prima delle aggregazioni.
    matrix_df = crea_matricione()

    # Parte B: analisi univariata.
    result_b = apply_univariate_outlier_detection(
        df=matrix_df,
        exclude_columns=EXCLUDE_COLUMNS,
        mad_threshold=3.5,
        min_observations=10,
    )

    # Parte C: analisi multivariata sul dataframe arricchito da Parte B.
    result_c = apply_dbscan_outlier_detection(
        df=result_b.dataframe,
        feature_set=DBSCAN_FEATURE_SET,
```

```
        eps=DBSCAN_EPS,
        min_samples=DBSCAN_MIN_SAMPLES,
        max_missing_rate=DBSCAN_MAX_MISSING_RATE,
    )

    # Salvataggio del matricione finale e dei report.
    result_c.dataframe.to_csv(MATRICIONE_OUTPUT, index=False)
    result_b.report.to_csv("report_univariato.csv", index=False)

    dbscan_report = pd.DataFrame(
        list(result_c.report.items()),
        columns=["parameter", "value"],
    )
    dbscan_report.to_csv("report_dbscan.csv", index=False)


if __name__ == "__main__":
    main()
```

24. Il flusso esatto tra i file

```
main.py
    |
    |-- importa crea_matricione da pipeline_filippo.py
    |
    |-- crea_matricione()
    |       |
    |       |-- carica dati elementari
    |       |-- richiama clean_raw_measurements()  [Parte A]
    |       |-- costruisce aggregazioni
    |       |-- restituisce matrix_df
    |
    |-- apply_univariate_outlier_detection(matrix_df)  [Parte B]
    |
    |-- apply_dbscan_outlier_detection(matrix_with_b)   [Parte C]
    |
    |-- salva matricione e report
```

In questo modo tu esegui solo:

```
python main.py
```

Il `main.py` coordina il lavoro, ma la logica rimane nei moduli specializzati.

OUTPUT E INTERPRETAZIONE

25. Colonne generate dalla Parte A

```
<feature>_invalid
n_invalid_values
has_invalid_value
```

Queste colonne documentano la presenza di valori impossibili nei dati elementari.

26. Colonne generate dalla Parte B

```
<feature>_univariate_outlier
<feature>_robust_zscore
n_univariate_outliers
has_univariate_outlier
```

Queste colonne indicano estremi statistici per singola feature.

27. Colonne generate dalla Parte C

```
dbscan_evaluable
dbscan_cluster
dbscan_low_density
```

Queste colonne indicano se la riga è stata valutata e se appartiene a una regione DBSCAN a bassa densità.

28. Uso delle flag

Le flag non devono essere interpretate automaticamente come diagnosi o errori clinici. Servono a:

- documentare la qualità dei dati;
- identificare osservazioni da controllare;
- confrontare analisi con e senza casi rari;
- verificare la stabilità dei modelli;
- descrivere il preprocessing nel report finale.

Esempio di conteggio:

```
n_invalid = final_matrix["has_invalid_value"].sum()
n_univariate = final_matrix["has_univariate_outlier"].sum()
n_dbscan = final_matrix["dbscan_low_density"].sum()
```

29. Flag e modelli predittivi

Le flag non devono essere inserite automaticamente tra le feature del modello. Sono indicatori di qualità e rarità, non necessariamente variabili cliniche predittive.

Possono essere usate per analisi di sensibilità:

```
model_all = train_model(final_matrix)

model_without_dbscan_noise = train_model(
    final_matrix[final_matrix["dbscan_low_density"] != 1]
)
```

Il confronto deve essere documentato e non deve trasformarsi in una cancellazione arbitraria dei pazienti.

DECISIONI PROGETTUALI

30. Perché due momenti diversi

La Parte A lavora sui dati elementari e deve precedere le aggregazioni. Le Parti B e C lavorano sul matricione finale.

Unire tutto in una sola funzione sarebbe poco adatto al flusso reale: Filippo deve usare A prima delle aggregazioni, mentre tu devi usare B e C dopo.

31. Perché i limiti sono in un file separato

`physiological_ranges.py` contiene configurazione, non logica. Se cambiano un'unità, il nome di una colonna o una soglia, si modifica solo quel file.

Questo riduce il rischio di rompere le funzioni di pulizia o di DBSCAN.

32. Perché le funzioni sono in un modulo separato

`outlier_pipeline.py` contiene la logica operativa. Il `main.py` la richiama senza duplicarla.

Vantaggi:

- codice più leggibile;
- funzioni riutilizzabili;
- meno copia-incolla;
- integrazione più semplice con il codice di Filippo;
- parametri centralizzati e modificabili.

33. Perché non si eliminano righe

Una riga rappresenta un paziente o un ricovero. Un valore raro può essere reale e informativo. La pipeline conserva quindi tutte le righe e aggiunge flag.

La decisione di escludere casi, imputare missing o cambiare il dataset deve essere presa in una fase separata, documentata e condivisa dal gruppo.

34. Perché non si usa un numero fisso di feature

Non si dice semplicemente “usiamo 12 feature”. Il numero dipende da:

- significato clinico;
- disponibilità delle colonne;
- missingness;
- momento temporale;
- interpretabilità;
- stabilità del clustering.

Si parte dal set `triage`, poi si confrontano configurazioni più ricche come `triage_plus_blood`.

INSTALLAZIONE E UTILIZZO

35. Installazione

Dalla cartella del progetto:

```
pip install pandas numpy scikit-learn
```

36. Import da notebook o altro script

```
from outlier_functions.outlier_pipeline import (
    clean_raw_measurements,
    apply_univariate_outlier_detection,
    apply_dbscan_outlier_detection,
)
```

Per i limiti:

```
from outlier_functions.physiological_ranges import (
    PHYSIOLOGICAL_RANGES_TRIAGE,
    PHYSIOLOGICAL_RANGES_BLOOD,
    DEFAULT_PHYSIOLOGICAL_RANGES,
    FEATURE_SETS,
)
```

37. Esecuzione

```
python main.py
```

Il codice di Filippo deve essere importabile, per esempio con:

```
from pipeline_filippo import crea_matricione
```

Se il suo file ha un nome o una funzione diversa, nel `main.py` bisogna sostituire solo quella riga e la relativa chiamata.

CHECKLIST FINALE

Prima di usare la pipeline sui dati reali:

- verificare le unità di temperatura;
- verificare i nomi reali delle colonne;
- verificare le unità degli esami del sangue;
- inserire la Parte A prima delle aggregazioni di Filippo;
- controllare che Filippo usi i dataframe puliti;
- verificare le feature presenti nel matricione;
- escludere identificativi, target, date e leakage dalla Parte B;
- iniziare DBSCAN con il set `triage`;
- controllare quante righe sono valutabili;
- calibrare `eps` e `min_samples` sui dati reali;
- salvare i report di Parte A, Parte B e Parte C;
- usare le flag per controllo e analisi di sensibilità, non per eliminazioni automatiche.

Sintesi operativa

```
Filippo:
    dati elementari
    -> clean_raw_measurements()       Parte A
    -> aggregazioni
    -> crea_matricione()

Tu nel main.py:
    matricione
    -> apply_univariate_outlier_detection()   Parte B
    -> apply_dbscan_outlier_detection()       Parte C
    -> salvataggio risultati
```

Questa è la logica definitiva del progetto: Parte A nel flusso di Filippo, Parti B e C nel tuo `main.py`, configurazioni separate e funzioni riutilizzabili.